Does More Thinking Always Help?

Prompt Strength, Reasoning Length, and Model Size in Small-to-Mid-Scale LLMs

Weihao Li Shijie Chen Katie Shao Yunfei Ge Di Hu

CS 396 — Reasoning and Planning in the Foundation Model Era

Northwestern University, Winter 2026

Abstract

Recent reasoning-oriented models suggest a simple rule of thumb for inference: giving a model more room to think leads to better answers. However, this claim is mostly supported by frontier-scale systems, and it remains unclear whether the same pattern holds for the smaller instruction-tuned models commonly used in practice.

We study two complementary forms of inference-time reasoning control: *explicit step budgets* and *prompt strength*. Across Qwen-family models ranging from 1.5B to 14B parameters, we evaluate fixed reasoning-step conditions and prompt levels of varying strength on mathematical reasoning (GSM8K) and reading comprehension (DROP).

Our results show that neither longer reasoning nor stronger prompting is universally beneficial. For explicit step budgets, small models often fail to follow long reasoning instructions, mid-size models exhibit a non-monotonic sweet spot, and only the largest models benefit consistently from longer reasoning. For prompt design, the best prompt is task-dependent: on GSM8K, moderate prompt strength tends to work best, while on DROP, stronger and more constrained prompts often improve both accuracy and efficiency. Overall, the value of inference-time reasoning control depends jointly on model size, task structure, and the specific control mechanism being applied.

1 Introduction

1.1 Background and Motivation

The premise behind chain-of-thought (CoT) prompting is intuitive: models perform better on complex tasks when they spell out their intermediate steps [12]. This basic idea has evolved rapidly. Adding "Let's think step by step" enables zero-shot CoT [6], while sampling multiple paths [11] makes the reasoning more robust. Recently, systems like OpenAI o1 [9] and DeepSeek-R1 [4] have strengthened the broader narrative that scaling up inference-time "thinking" can drive major gains on difficult benchmarks.

This creates an appealing assumption: more test-time reasoning should generally improve performance. But most evidence for that assumption comes from very large models. For the smaller instruction-tuned models that are much more common in local deployment and resource-constrained settings, the picture may be more complicated.

1.2 The Gap

Most existing work discusses test-time reasoning as if it were a single knob: let the model think longer, and performance should improve. In practice, however, there are at least two distinct ways to control reasoning at inference time. One is to directly constrain the *length of reasoning* by forcing a fixed number of intermediate steps. The other is to alter the *prompt itself*, changing how strongly the model is encouraged or constrained to reason.

These two controls are often conflated, even though they may affect model behavior in very different ways. Moreover, prior evidence comes largely from very large models. What is still missing is a systematic study of how *step budget* and *prompt strength* interact with model scale in the 1.5B–14B range that is much more representative of practical deployment.

1.3 Research Questions

RQ1. Does increasing an explicit reasoning-step budget monotonically improve accuracy across model sizes?

RQ2. Does increasing prompt strength monotonically improve accuracy across model sizes?

RQ3. Do these two forms of inference-time control behave differently across mathematical reasoning (GSM8K) and reading comprehension (DROP)?

RQ4. Is there a capability threshold below which explicit reasoning control becomes ineffective or harmful?

1.4 Contributions

We make two main contributions. First, we separate two commonly conflated aspects of inference-time reasoning control: *step budget* and *prompt strength*. Rather than treating "more reasoning" as a single variable, we evaluate them independently.

Second, we show that the effectiveness of these controls is strongly conditional. Explicit multi-step reasoning helps only once models are large enough to sustain coherent reasoning, and even then it can become harmful past a task-dependent sweet spot. Prompt design is equally non-universal: moderate prompts work best on GSM8K, while stronger and more constrained prompts are often more effective on DROP. Together, these results argue against any one-size-fits-all prompting strategy.

2 Related Work

2.1 Chain-of-Thought Prompting

Wei et al. [12] showed that intermediate reasoning steps unlock performance on complex tasks. This was quickly adapted into zero-shot settings [6] and stabilized via self-consistency voting [11].

2.2 Test-Time Compute Scaling

OpenAI o1 [9] and DeepSeek-R1 [4] showed that scaling inference compute translates directly to capability. Snell et al. [10] formalized this tradeoff, suggesting test-time compute can substitute for model parameters. Muennighoff et al. [8] introduced budget forcing to actively cap reasoning tokens during generation.

2.3 CoT Length Effects

Jin et al. [5] found that extending CoT steps improves GPT-3.5 and GPT-4 even if intermediate steps contain flaws. Chen et al. [1] theoretically mapped an inverted U-shaped curve for CoT length and noted a "simplicity bias" in stronger models. Li et al. [7] observed "Long CoT Degradation" during training, where small models fine-tuned on long traces lost up to 75% of their performance.

2.4 Prompt Design and Reasoning Control

A related line of work studies how prompting format itself shapes model behavior. Even without fixing a step budget, prompts can strongly influence whether a model reasons explicitly, answers tersely, or over-produces irrelevant text. Our work adds to this line by showing that prompt design and explicit step forcing are not interchangeable controls: they produce different patterns across model sizes and task types.

2.5 Our Position

Our work bridges these threads by empirically testing two forms of inference-time control across small-to-mid-scale models. While much prior work focuses either on very large models or on training-time effects, we isolate the immediate effect of longer explicit reasoning and stronger prompting across scales and task types.

3 Methodology

3.1 Models

We evaluate five instruction-tuned models from the Qwen family, spanning roughly two orders of magnitude in size (Table 1). Sticking to one model family minimizes confounds from different pre-training recipes or architectures, making parameter count the primary driver of any observed differences.

Table 1: Models evaluated in this study.

Model	Params	Family
Qwen2.5-1.5B-Instruct	1.5B	Qwen2.5
Qwen3-4B-Instruct	4B	Qwen3
Qwen2.5-7B-Instruct	7B	Qwen2.5
Qwen2.5-14B-Instruct	14B	Qwen2.5

3.2 Datasets

GSM8K [2] contains 8.5K grade-school math word problems requiring multi-step arithmetic. We use the standard test split.

DROP [3] contains reading comprehension questions requiring discrete operations such as counting, comparison, and arithmetic over paragraphs. We use the validation split. These datasets present structurally different challenges: GSM8K naturally decomposes into sequential math steps, whereas DROP requires locating relevant text spans before making inferences.

3.3 Inference-Time Control Conditions

We study two forms of inference-time control.

Step-budget experiment. In the first setting, we explicitly enforce a fixed reasoning structure across four conditions: direct, short, medium, and long. For GSM8K, these correspond to target step budgets of 0, 2, 4, and 8, paired with maximum generation budgets of 64, 128, 256, and

512 new tokens. Models are instructed to either answer directly or write exactly the specified number of steps, each marked with `Step k:`, followed by a final line in the format `Final Answer: <number>`.

Prompt-level experiment. In the second setting, we vary prompt strength from level 0 to level 4. These prompts differ in how strongly they encourage structured reasoning and answer formatting, ranging from minimal prompting to increasingly explicit reasoning-oriented instructions. Unlike the step-budget experiment, prompt levels do not force an exact number of steps; instead, they influence reasoning behavior indirectly through prompt design.

Table 2: Step-budget conditions used in the explicit reasoning experiment.

Condition	Steps	Max Tokens	Description
Direct	0	64	Answer only, no reasoning
Short	2	128	2 explicit steps
Medium	4	256	4 explicit steps
Long	8	512	8 explicit steps

Table 3: Prompt-level conditions used in the prompt-design experiment.

Prompt Level	Description
Level 0	Minimal prompt, little or no reasoning guidance
Level 1	Light reasoning cue
Level 2	Moderate reasoning-oriented prompt with explicit final answer format
Level 3	Stronger reasoning guidance and constraints
Level 4	Most explicit and structured prompt in the set

3.4 Decoding and Parsing Controls

All experiments use greedy decoding with `temperature=0` and `top_p=1.0`. When available, we use the model tokenizer's chat template for system/user formatting; otherwise, we fall back to plain-text prompting. We log input tokens, output tokens, and wall-clock latency for every example.

For GSM8K, all prompt-level conditions require the final prediction to appear in the format `Final Answer: <number>`. During evaluation, we first extract the answer from that line; if it is missing, we fall back to the last numeric string appearing in the output. In the step-budget setting, we additionally estimate realized reasoning length by counting generated step markers of the form `Step k:`.

3.5 Evaluation Protocol

We measure Exact Match (EM) following standard practices for GSM8K and DROP. Answers are lowercased and stripped of punctuation as applicable. For each condition, we sample 100 examples (`seed=42`) and use deterministic decoding.

For the step-budget experiments, we compare the target step count with the realized count (`steps_est`) to evaluate instruction compliance. For the prompt-level experiments, where step counts are not explicitly constrained, we use prompt level as the primary independent variable and analyze token usage and latency as auxiliary signals of how much reasoning the model actually produced.

4 Results

4.1 Step-Budget Results on GSM8K

Table 4 shows that explicit reasoning length interacts strongly with model size.

Table 4: GSM8K Exact Match (%) by model and explicit step-budget condition. Bold = best per model. Dashes indicate conditions not run.

Model	Dir.	Sh.(2)	Med.(4)	Lo.(8)	Δ
Qwen2.5-1.5B	9.0	15.0	17.0	**22.0**	+13
Qwen3-4B	30.0	75.0	**91.0**	86.0	+61
Qwen2.5-7B	—	—	—	69.0	—
Qwen2.5-14B	33.0	69.0	88.0	**93.0**	+60

The Capability Threshold. The smallest models do not reliably convert extra step budget into useful reasoning. Qwen2.5-1.5B improves from 9% to 22%, but its absolute accuracy remains low.

The Non-Monotonic Sweet Spot. Qwen3-4B exhibits the clearest inverted-U pattern. Accuracy rises sharply from direct answering to 4 steps, peaking at 91%, but then falls to 86% at 8 steps. This suggests that once reasoning becomes too long, the model begins to introduce unnecessary or error-prone intermediate computations.

Monotonic Scaling in the Largest Model. Only Qwen2.5-14B follows the textbook test-time scaling story. Its performance improves steadily from 33% to 93% as the step budget increases.

Table 5: Instruction compliance on GSM8K: fraction of outputs that contain exactly the target number of steps.

Model	Sh.(2)	Med.(4)	Lo.(8)
Qwen2.5-1.5B	79%	70%	47% (avg 6.91)
Qwen3-4B	100%	100%	100%
Qwen2.5-7B	—	—	54% (avg 7.49)
Qwen2.5-14B	99%	92%	97%

Step Fidelity Matters. The compliance numbers show a scale-dependent behavioral difference. Qwen3-4B and Qwen2.5-14B almost perfectly follow the requested step counts, while smaller models drift increasingly far from the target under long-budget prompts. This means poor small-model performance is not just a reasoning issue; it is also an instruction-following issue.

Reasoning Cost. Longer step budgets clearly cost more. For example, Qwen3-4B rises from roughly 0.9s in direct mode to over 16s at 8 steps, even though its best accuracy is already reached at 4 steps. For Qwen2.5-14B, the jump in latency is accompanied by real gains, but for smaller models the extra compute is often not worth it.

4.2 Prompt-Level Results on GSM8K

Unlike the step-budget experiment, the prompt-level experiment reveals that prompt strength is not monotonic and cannot be reduced to "longer is better."

Table 6: GSM8K Exact Match (%) by model and prompt level. Bold = best per model.

Model	L0	L1	L2	L3	L4
Qwen2.5-1.5B	41.0	41.0	**43.0**	9.0	27.0
Qwen2.5-7B	**47.0**	43.0	24.0	46.0	28.0
Qwen2.5-14B	61.0	58.0	**82.0**	50.0	62.0

Moderate Prompting Works Best on Math. For GSM8K, the strongest prompt is not the best prompt. Qwen2.5-14B reaches its highest score at level 2, rising from 61% to 82%, but then drops again at levels 3 and 4. Qwen2.5-1.5B shows only a small gain at level 2, and Qwen2.5-7B actually degrades substantially under levels 2 and 4.

Prompt Design Changes Model Behavior. The effect of prompt level appears to be mediated partly through output length. Some prompts induce long reasoning chains, while others compress the output dramatically.

Table 7: Average output tokens on GSM8K by prompt level.

Model	L0	L1	L2	L3	L4
Qwen2.5-1.5B	233.0	225.1	171.8	20.7	170.2
Qwen2.5-7B	232.4	234.0	7.4	64.8	7.2
Qwen2.5-14B	215.5	231.2	180.5	65.5	101.1

For Qwen2.5-14B, level 2 preserves substantial reasoning length and yields the best accuracy. For Qwen2.5-7B, however, levels 2 and 4 collapse the output to roughly 7 tokens and also produce the worst performance. In other words, some prompts do not merely guide reasoning; they over-constrain it.

Efficiency Trade-Off. Prompt engineering also affects latency. Some GSM8K prompts produce very fast generations, but this speed can come at large accuracy cost. For math reasoning, the best prompt is therefore not the cheapest one but the one that balances reasoning depth with control.

4.3 Step-Budget Results on DROP

The DROP step-budget results differ substantially from GSM8K, especially for smaller models.

Table 8: DROP Exact Match (%) by model and explicit step-budget condition. Bold = best per model.

Model	Dir.	Sh.(2)	Med.(4)	Lo.(8)
Qwen2.5-1.5B	**32.0**	26.0	22.0	21.0
Qwen2.5-7B	46.0	50.0	50.0	**56.0**
Qwen2.5-14B	54.0	54.0	64.0	**72.0**

Long Reasoning Hurts Small Models on Comprehension. Qwen2.5-1.5B degrades as soon as explicit step forcing is introduced, falling from 32% direct-answer accuracy to 21% at 8 steps. Unlike GSM8K, where longer reasoning provided at least a small benefit to 1.5B, on DROP it appears actively harmful.

Large Models Benefit from Explicit Steps. Qwen2.5-14B shows stable gains from 54% to 72% as step budget increases. Qwen2.5-7B improves more modestly, suggesting that explicit reasoning can help reading comprehension, but only once the model has enough capacity to sustain it.

Table 9: Instruction compliance on DROP: fraction of outputs that contain exactly the target number of steps.

Model	Sh.(2)	Med.(4)	Lo.(8)
Qwen2.5-1.5B	82%	69%	8%
Qwen2.5-7B	99%	83%	36%
Qwen2.5-14B	100%	99%	97%

Following the Instruction is Part of the Challenge. On DROP, just as on GSM8K, larger models are much better at obeying the requested step budget. Qwen2.5-1.5B reaches an average of only about 5.2 steps under the 8-step condition, while Qwen2.5-14B stays very close to the requested target. Thus, when small models fail under long-step prompting, they often fail both in reasoning quality and in formatting fidelity.

Cost vs. Value. The cost increase is again substantial. For Qwen2.5-14B, latency rises sharply under long-step prompting, but this increase is paired with real gains. For Qwen2.5-1.5B, however, cost rises while accuracy falls, making long explicit reasoning a poor trade-off on this task.

4.4 Prompt-Level Results on DROP

The prompt-level experiment on DROP exhibits the opposite trend from GSM8K.

Table 10: DROP Exact Match (%) by model and prompt level. Bold = best per model.

Model	P0	P1	P2	P3	P4
Qwen2.5-1.5B	10.0	10.0	30.0	20.0	**31.0**
Qwen2.5-7B	46.0	49.0	50.0	47.0	**52.0**
Qwen2.5-14B	58.0	64.0	51.0	65.0	**69.0**

Stronger Prompts Help on DROP. Unlike GSM8K, stronger prompt levels generally help the medium and large models on DROP. Qwen2.5-14B climbs from 58% at P0 to 69% at P4, while Qwen2.5-7B also reaches its best performance at P4.

Shorter Outputs Can Be Better. DROP prompt gains are accompanied by shorter generations, not longer ones.

Table 11: Average output tokens on DROP by prompt level.

Model	P0	P1	P2	P3	P4
Qwen2.5-1.5B	27.73	14.33	8.22	11.46	9.84
Qwen2.5-7B	9.81	9.60	8.03	5.62	8.09
Qwen2.5-14B	42.91	12.74	10.78	8.90	8.48

This is the reverse of the GSM8K pattern. On DROP, better prompts tend to produce shorter and more constrained answers. That makes sense given the task structure: many DROP items resemble span extraction or short numeric responses rather than open-ended multi-step derivations.

Prompting Can Improve Formatting, Not Just Reasoning. A key caveat is that DROP appears highly sensitive to answer formatting. In several cases, the model generated a sentence containing the correct gold answer but was still marked wrong because the output was not normalized as a short span or number. This means part of the apparent prompt gain on DROP may reflect better answer formatting and extraction rather than better internal reasoning alone.

A Different Prompting Regime by Task. Together, these results suggest that the best prompting strategy depends strongly on task type. GSM8K benefits from an intermediate level of explicit reasoning guidance; DROP benefits more from concise and constrained answer formatting.

4.5 Cross-Task Comparison

Table 12 summarizes the main contrast.

Table 12: Cross-task comparison summary.

Metric	GSM8K	DROP
Best prompt pattern	Moderate prompt	Strong prompt
Small-model step effect	Slight gain / unstable	Often harmful
Mid-size behavior	Sweet spot / mixed	Moderate gains
Largest-model step effect	Monotonic gain	Monotonic gain
Best reasoning regime	"Just enough" reasoning	Constrained answer generat

Math naturally decomposes into sequential steps, so some explicit reasoning is genuinely useful. Reading comprehension, by contrast, often depends on locating and formatting the correct span. For that reason, longer reasoning is not always beneficial, and stronger prompts may help mainly by making the final answer easier to extract and score.

5 Discussion

5.1 Inference-Time Control Is Not One-Dimensional

Our results challenge the idea that inference-time reasoning can be summarized by a single principle such as "longer is better." There are at least two distinct controls: how much reasoning the model is explicitly required to produce, and how strongly the prompt shapes its reasoning behavior. These controls do not have the same effect.

5.2 Why Step Budget and Prompt Strength Behave Differently

Explicit step budgets directly increase the amount of reasoning text that the model must produce. This can help capable models, but it also increases the chance of cascading error and off-task filler. Prompt strength works more indirectly: it changes the model's style of answering, sometimes eliciting useful reasoning, but other times suppressing it or over-constraining the output.

This explains why GSM8K and DROP behave so differently. On GSM8K, excessive compression hurts because multi-step arithmetic often requires visible intermediate computation. On DROP, shorter and more constrained answers can help because the task rewards precise extraction and formatting more than long-form derivation.

5.3 Why Mid-Size Models Hit a Sweet Spot

The clearest example is Qwen3-4B on GSM8K. Because it follows the requested step budget almost perfectly, the decline from 91% at 4 steps to 86% at 8 steps cannot be blamed on non-compliance. Instead, it suggests a true over-reasoning effect: once the model is forced to continue beyond the natural solution depth, it begins to add harmful intermediate content.

A similar "just enough reasoning" pattern also appears in the prompt-level results. On GSM8K, the best prompts are not the longest or strongest ones, but the ones that induce a moderate amount of useful reasoning without collapsing into either verbosity or over-compression.

5.4 Why Small Models Fail Differently Across Tasks

For smaller models, failure modes are task-specific. On GSM8K, longer reasoning sometimes provides a slight gain, but only because even noisy intermediate computation can occasionally help arithmetic problems. On DROP, the same models often do better by answering directly. Forcing long reasoning seems to disrupt simple retrieval-like behavior and pull the model away from the source passage.

5.5 Practical Implications

These findings matter for deployment. For small and medium local models, simply asking for more reasoning can waste compute or even reduce accuracy. On math tasks, moderate reasoning budgets or mid-strength prompts may be optimal. On extraction-heavy reading tasks, stronger prompts may help mostly by enforcing concise, scoreable outputs. In short, efficient prompting should be matched to both the model and the task rather than copied from frontier-model recipes.

5.6 Connection to Course Themes

This project grounds several CS 396 themes in empirical evidence. It refines the test-time compute scaling story by showing that extra reasoning is only useful when the model can actually exploit it. It also highlights an alignment-style question: rather than forcing all models to reason longer, future systems may need to learn when to stop. In that sense, our findings connect naturally to adaptive inference and reinforcement learning approaches that allocate reasoning budget dynamically instead of using a fixed prompting rule.

6 Limitations

Several constraints shape our findings:

Incomplete experiment grid. We only ran the 1.5B, 7B, 14B models on the long condition for GSM8K in the step-budget setup. Full curves would more precisely identify where the capability threshold begins.

Single model family. We only tested Qwen models. Different architectures, alignment recipes, or tokenizer behaviors could shift the observed sweet spots.

Prompt strength vs. prompt length. Our prompt-level experiment does not vary raw prompt length alone. The five prompt levels differ jointly in verbosity, explicitness, reasoning cues, and output-format constraints. Therefore, we interpret this factor as *prompt strength* rather than pure input length.

Imperfect step estimation. In the prompt-level experiments, visible reasoning length is only indirectly reflected by token counts or structural traces. In the step-budget experiments, `steps_est` measures compliance with explicit `Step k:` markers, not necessarily genuine internal reasoning quality.

Answer extraction heuristics. Our GSM8K evaluation relies on rule-based parsing. We first parse the line beginning with `Final Answer:`; if that fails, we fall back to

the last numeric string in the generation. This improves robustness but may still mis-score generations with unusual formatting.

Formatting sensitivity on DROP. Some DROP results appear affected by answer normalization. A prediction that semantically contains the correct answer can still be scored as incorrect if it is wrapped in a full sentence. Therefore, part of the prompt effect on DROP likely reflects output formatting rather than reasoning quality alone.

Coarse control grid. For step budgets, testing only a small set of fixed values means the true optimum may lie in between. Likewise, prompt levels 0–4 are coarse categories rather than a continuous scale.

7 Conclusion

The idea that more thinking always leads to better performance is appealing, but our results show that it is incomplete. Inference-time reasoning control depends not only on how much reasoning is requested, but also on how that reasoning is elicited.

Across both GSM8K and DROP, we find that explicit step budgets and prompt strength behave differently. For step forcing, small models often fail to benefit and sometimes degrade, while larger models benefit more reliably. For prompt design, the best strategy is strongly task-dependent: GSM8K favors moderate reasoning-oriented prompts, whereas DROP tends to favor stronger and more constrained prompts that produce short, scoreable answers.

Overall, there is no single prompting strategy that works best across all models and tasks. Effective inference-time control must be matched to model capacity, task structure, and the specific mechanism used to shape reasoning.

8 Future Work

A natural next step is to fill the missing parts of the experiment grid, especially for the 1.5B and 14B GSM8K step-budget runs. Finer step budgets such as 3, 5, and 6 could identify the true optimum more precisely for mid-size models.

It would also be valuable to extend the same framework to other model families such as LLaMA or Mistral to test whether the same task-dependent patterns hold more broadly.

Finally, a stronger evaluation pipeline should separate *reasoning quality* from *answer formatting*. On tasks like DROP, some apparent prompt gains may come from improved extractability rather than better reasoning. A future study could use stronger answer normalization, process-based metrics, or step-level judging to better disentangle those effects.

References

[1] X. Chen et al. (2025). When More is Less: Understanding Chain-of-Thought Length in LLMs. *ICLR 2026*. arXiv:2502.07266.

[2] K. Cobbe et al. (2021). Training Verifiers to Solve Math Word Problems. arXiv:2110.14168.

[3] D. Dua et al. (2019). DROP: A Reading Comprehension Benchmark Requiring Discrete Reasoning Over Paragraphs. *NAACL 2019*.

[4] D. Guo et al. (2025). DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. arXiv:2501.12948.

[5] M. Jin et al. (2024). The Impact of Reasoning Step Length on Large Language Models. *ACL 2024 Findings*. arXiv:2401.04925.

[6] T. Kojima et al. (2022). Large Language Models are Zero-Shot Reasoners. *NeurIPS 2022*.

[7] Z. Li et al. (2025). Through the Valley: Path to Effective Long CoT Training for Small Language Models. arXiv:2506.07712.

[8] N. Muennighoff et al. (2025). s1: Simple Test-Time Scaling. arXiv:2501.19393.

[9] OpenAI (2024). Learning to Reason with LLMs. *OpenAI Blog*.

[10] C. Snell et al. (2024). Scaling LLM Test-Time Compute Optimally can be More Effective than Scaling Model Parameters. *ICLR 2025*. arXiv:2408.03314.

[11] X. Wang et al. (2023). Self-Consistency Improves Chain of Thought Reasoning in Language Models. *ICLR 2023*.

[12] J. Wei et al. (2022). Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. *NeurIPS 2022*.